

VIT-AP
UNIVERSITY

DIGITAL LOGIC DESIGN
(Course Code: ECE 1003)

Module-2:Lecture-4

Combinational Circuit Design

CONTENTS

Module-2

- ❖ Binary adder and subtractor
- ❖ Ripple carry adders/subtractors and fast adders
- ❖ Binary decoders, **encoders**
- ❖ **Multiplexers** and de-multiplexers
- ❖ Logic functions using decoders and multiplexers
- ❖ Code converters
- ❖ Magnitude comparator

CONTENTS

Lecture-4

- ❖ Encoder
- ❖ Priority Encoder
- ❖ Multiplexer

BOOKS

Textbooks

1. M.Morris Mano, Michael D Ciletti, **Digital Design**, 5th edition, Pearson Publishers, 2013.
2. R.P. Jain, “**Modern Digital Electronics**”, 4th edition, TMH.

References

1. M.Morris Mano, Charles R. Kime, Tom Martin, **Logic and Computer Design Fundamentals**, 4th edition, Pearson Publishers.
2. C. H. Roth and L. L. Kinney, **Fundamentals of Logic Design**, 5th edition, Cengage Publishers.

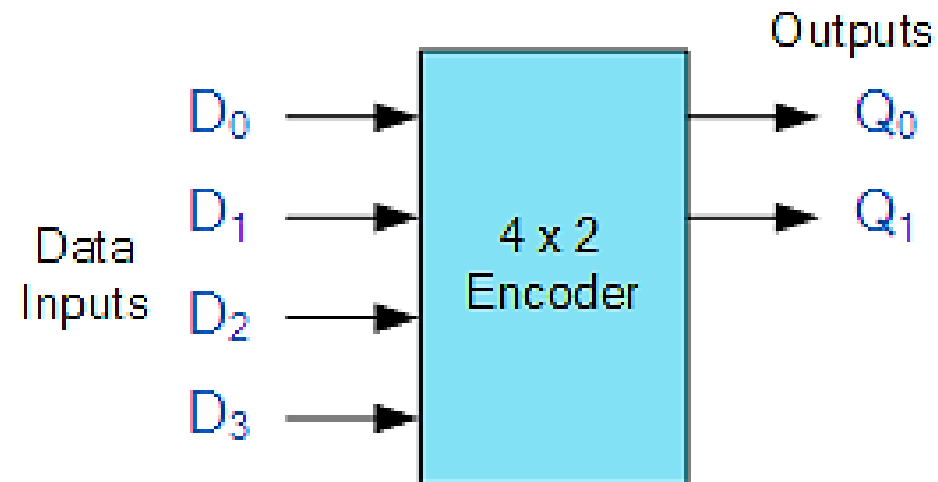
Encoder

ENCODER

- ✓ An **encoder** is a digital circuit that performs the **inverse operation of a decoder**.
- ✓ It has **2^n or fewer input lines and n output lines**.

4x2 Encoder

- ✓ It has **$2^2 = 4$** or fewer input lines and **3** output Lines.



Inputs				Outputs	
D_3	D_2	D_1	D_0	Q_1	Q_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1
0	0	0	0	x	x

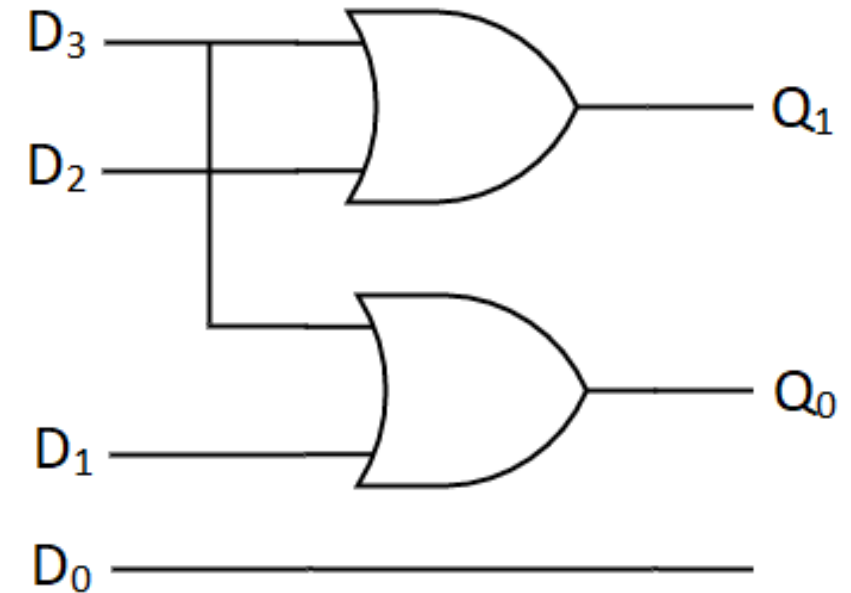
ENCODER

4x2 Encoder: Logic Diagram

- ✓ The **encoder** can be implemented with **OR gates** whose inputs are determined directly from the truth table.
- ✓ **Output Q_1** is equal to 1 when the **input digits are 2 or 3**.
- ✓ **Output Q_0** is 1 for **input digits 1, or 3**.
- ✓ These conditions can be expressed by the following Boolean output functions:

$$Q_1 = D_2 + D_3$$

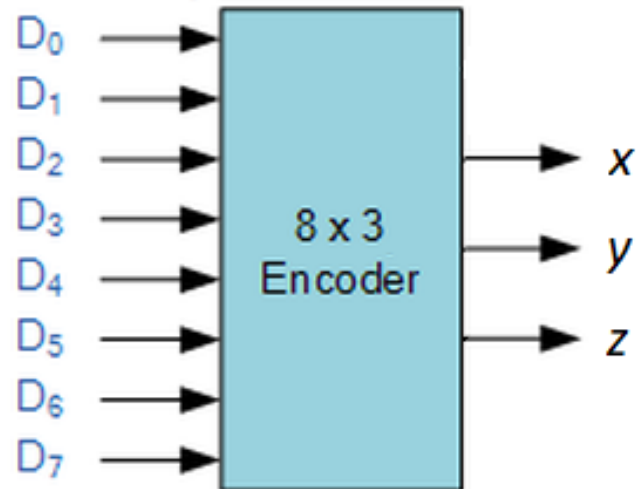
$$Q_0 = D_1 + D_3$$



ENCODER

8x3 Encoder

- ✓ It has $2^3 = 8$ or fewer input lines and 3 output lines.



Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

ENCODER

8x3 Encoder

- ✓ The encoder can be implemented with OR gates whose inputs are determined directly from the truth table.
- ✓ Output z is equal to 1 when the input octal digit is 1, 3, 5, or 7.
- ✓ Output y is 1 for octal digits 2, 3, 6, or 7, and output x is 1 for digits 4, 5, 6, or 7.
- ✓ These conditions can be expressed by the following Boolean output functions:

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

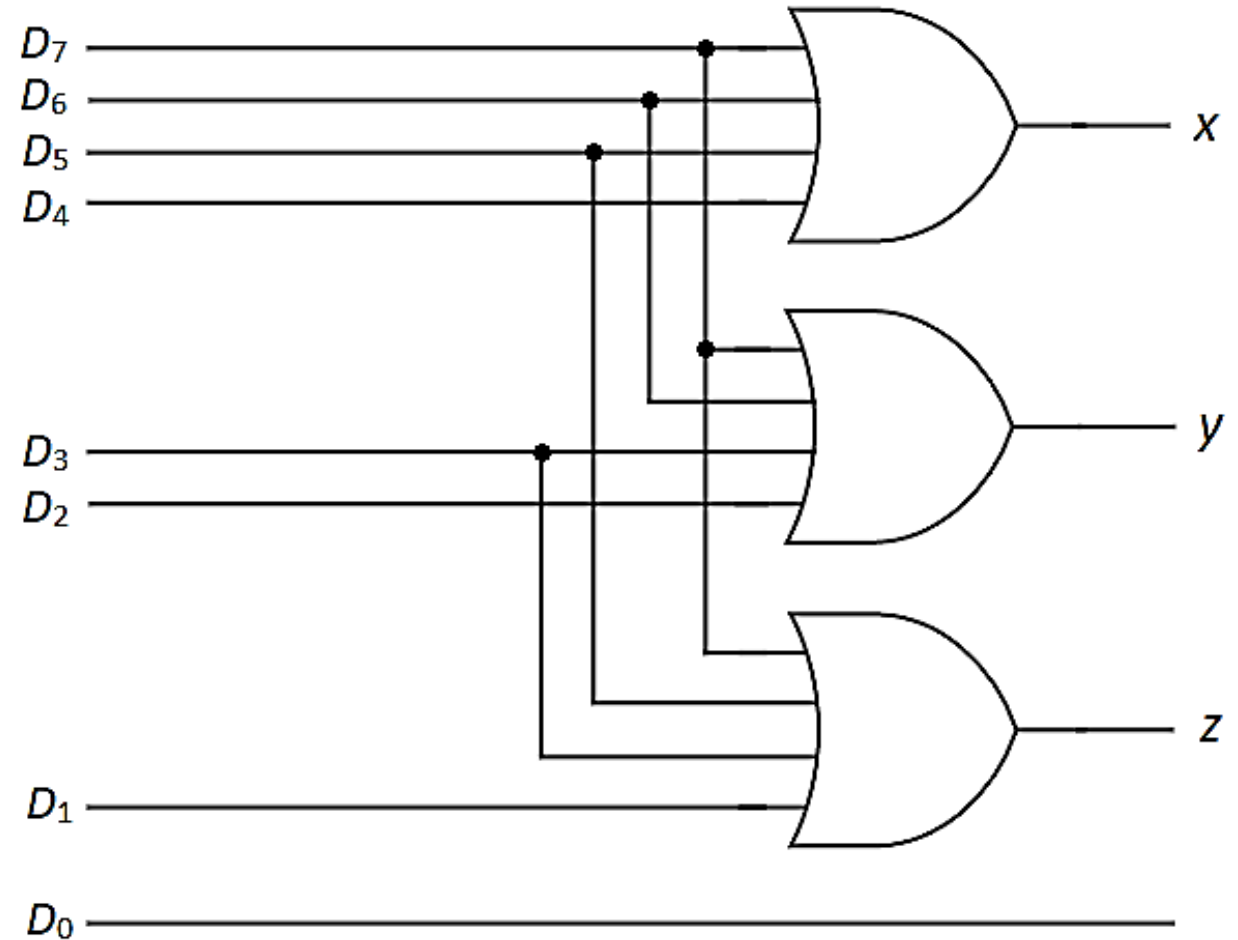
ENCODER

8x3 Encoder: Logic Diagram

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$



ENCODER

Limitations of Encoder

- ✓ The **encoder** defined has the **limitation** that only **one input can be active at any given time**.
- ✓ If **two inputs are active** simultaneously, the output produces an **undefined combination**.
- ✓ For example, if D_3 and D_6 are **1** simultaneously, the output of the encoder will be **111** because all three outputs are equal to **1**. The output **111** does not represent either binary **3** or binary **6**.
- ✓ To resolve this ambiguity, **encoder circuits must establish an input priority** to ensure that **only one input is encoded**. This encoder is called as **Priority Encoder**.
- ✓ If we establish **a higher priority** for inputs with **higher subscript numbers**, and if both D_3 and D_6 are **1** at the same time, the output will be **110** because D_6 has higher priority than D_3 .
- ✓ Another ambiguity in the octal-to-binary encoder is that **an output with all 0's is generated** when **all the inputs are 0**; but this output is the same as when D_0 is equal to **1**.
- ✓ The discrepancy can be **resolved by providing one more output to indicate** whether at least one input is equal to **1**.

ENCODER

Priority Encoder

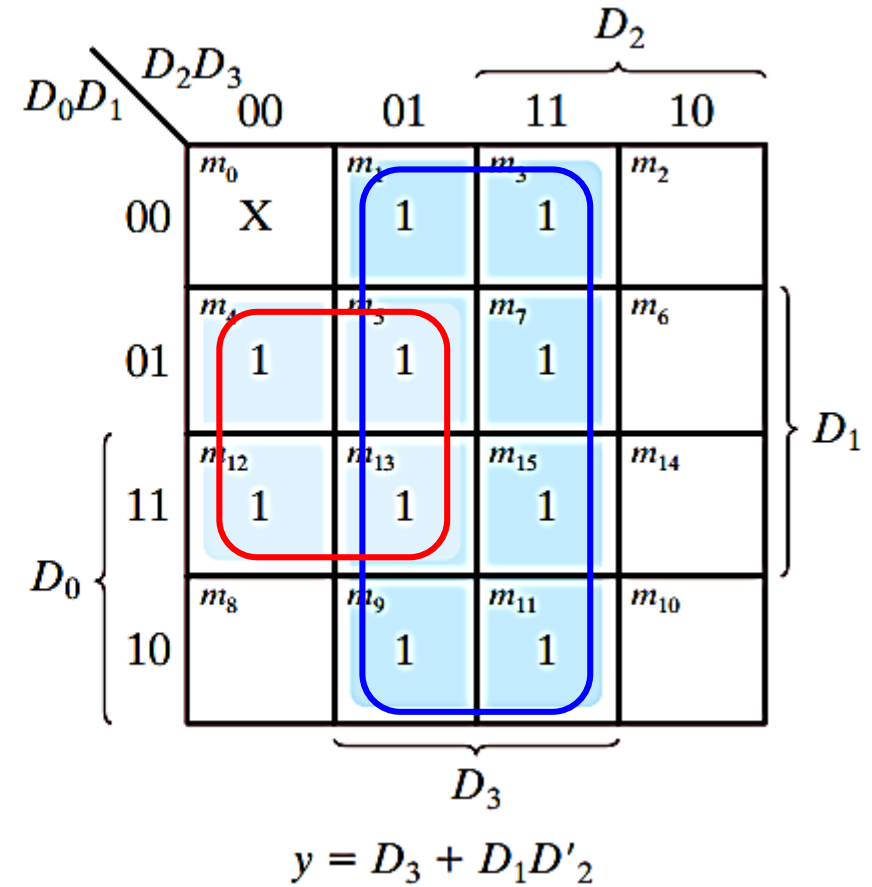
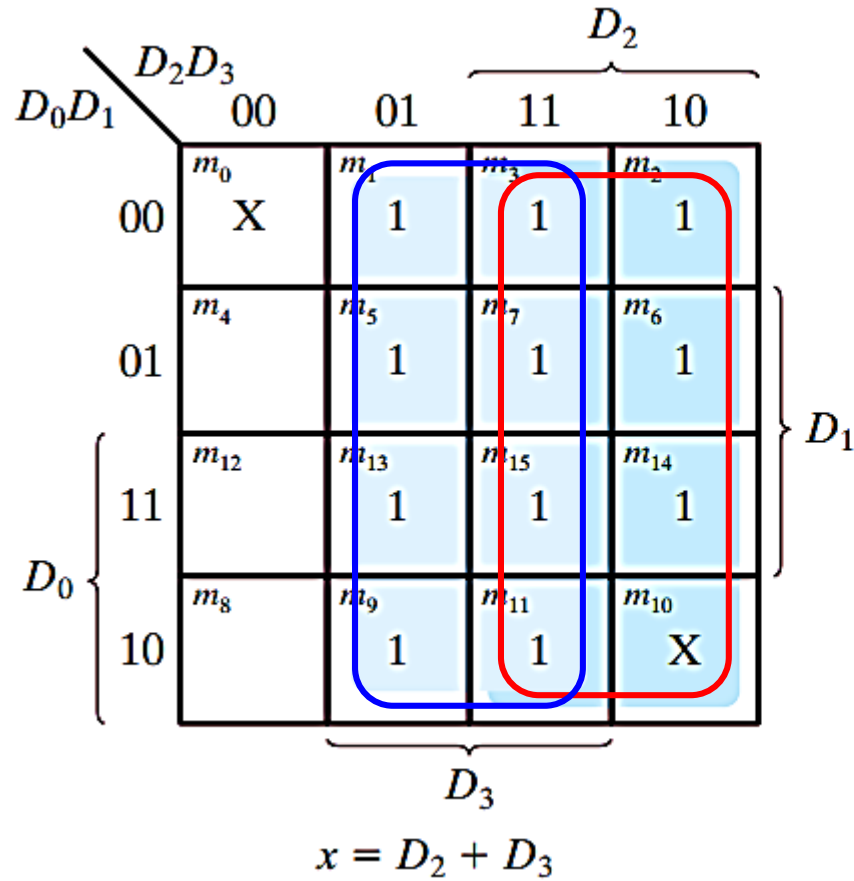
- ✓ A **priority encoder** is an encoder circuit that **includes the priority function**.
- ✓ The operation of the priority encoder is such that if **two or more inputs are equal to 1** at the same time, the input having the **highest priority will take precedence**.
- ✓ An example of **4x2 priority decoder** is to be designed with **valid bit output indicator**.
- ✓ A **valid bit indicator that is set to 1** when **one or more inputs are equal to 1**. If **all inputs are 0**, there is no valid input and **V is equal to 0**.

Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

ENCODER

Priority Encoder: Boolean Function Minimization



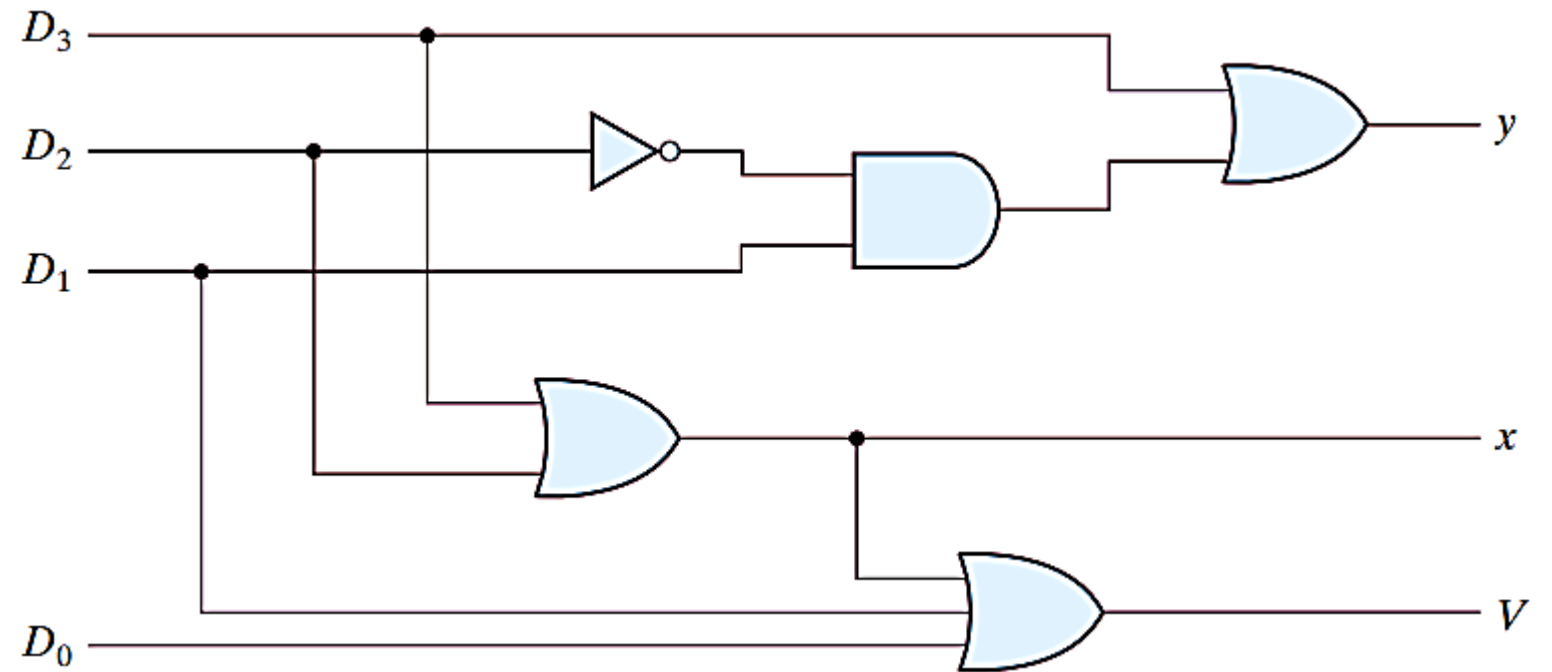
ENCODER

Priority Encoder: Logic Design

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

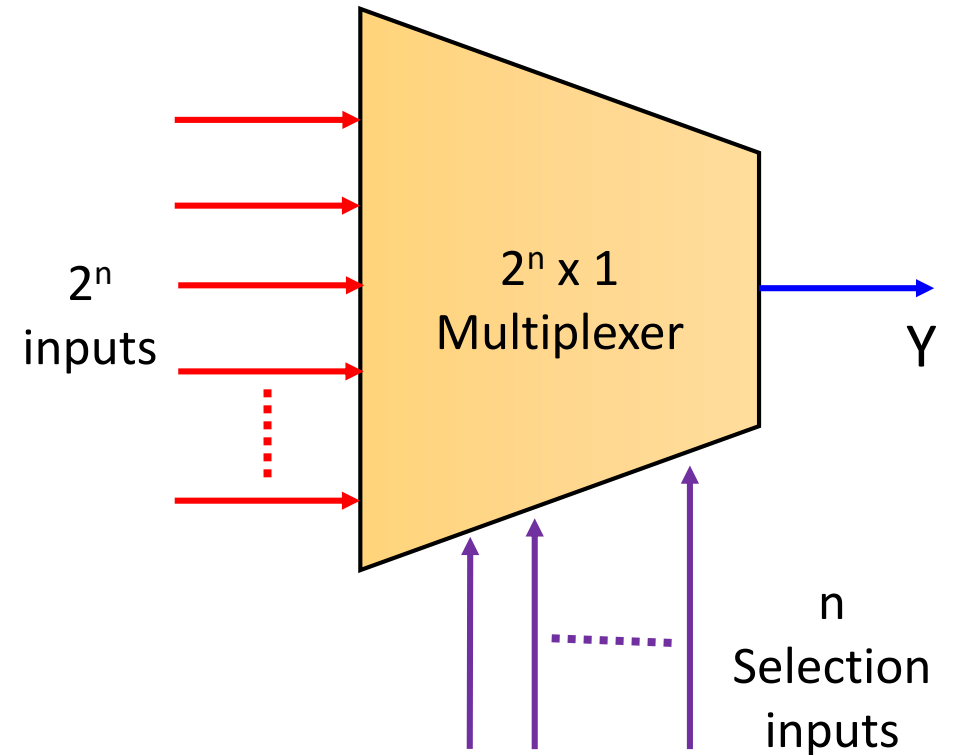
$$V = D_0 + D_1 + D_2 + D_3$$



Multiplexer

MULTIPLEXER

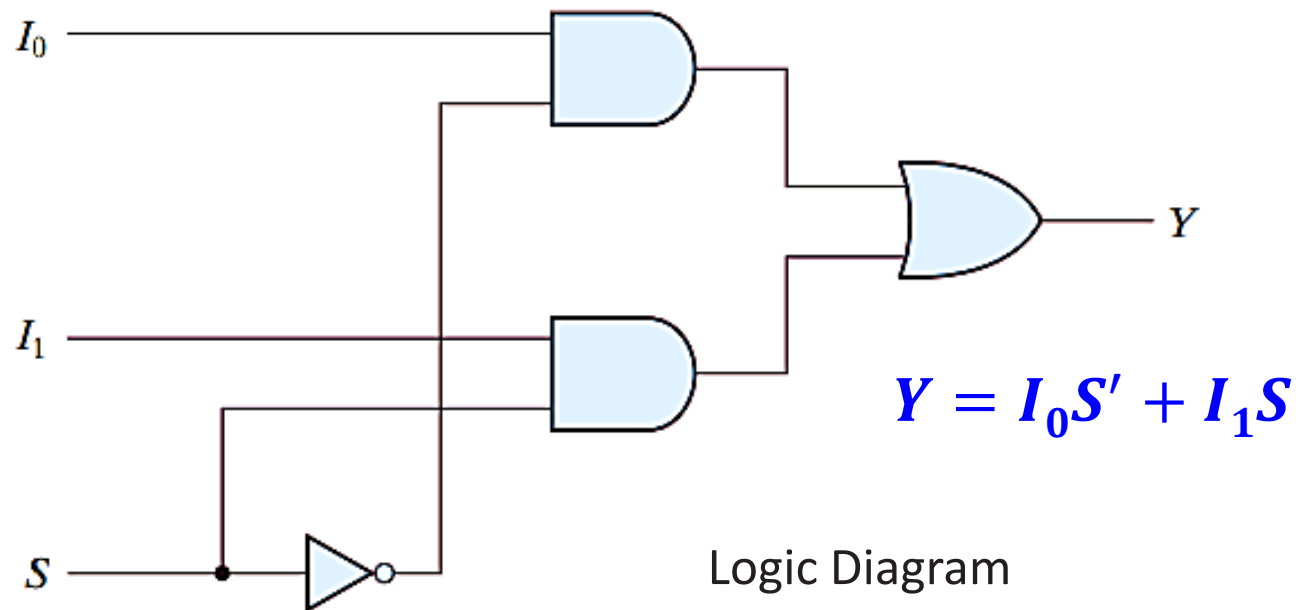
- ✓ A **multiplexer** is a combinational circuit that selects binary information from **one of many input lines and directs it to a single output line**.
- ✓ The **selection of a particular input line** is controlled by a set of **selection lines**.
- ✓ Normally, there are **2^n input lines** and **n selection lines** whose bit combinations determine which input is selected.
- ✓ A **multiplexer or MUX** is also known as a **data selector**.
- ✓ Multiplexers can also be used to implement Boolean functions of multiple variables.
- ✓ A **multiplexer** can take a form as **2×1 , 4×1 , 8×1 , 16×1 and so on**.



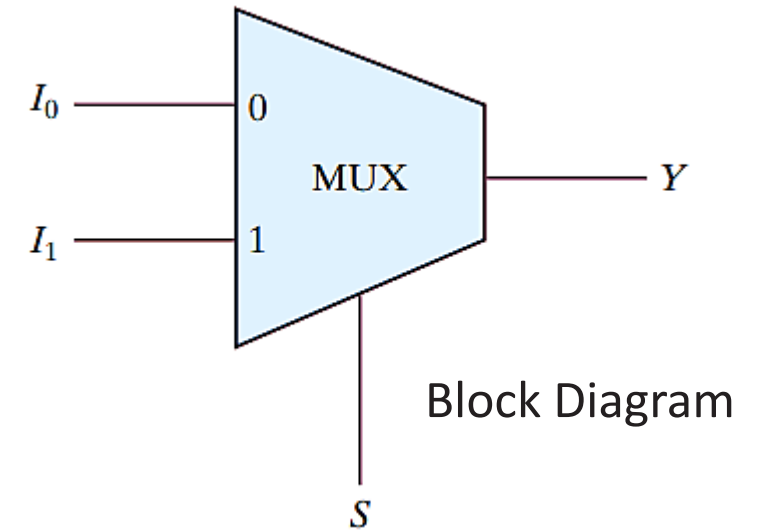
MULTIPLEXER

2x1 Multiplexer

- ✓ A **2x1 multiplexer** has $2 = 2^1$ inputs and one output.
- ✓ The input lines can be selected by **one selection input S**.



$$Y = I_0 S' + I_1 S$$



S	Y
0	I_0
1	I_1

Function table

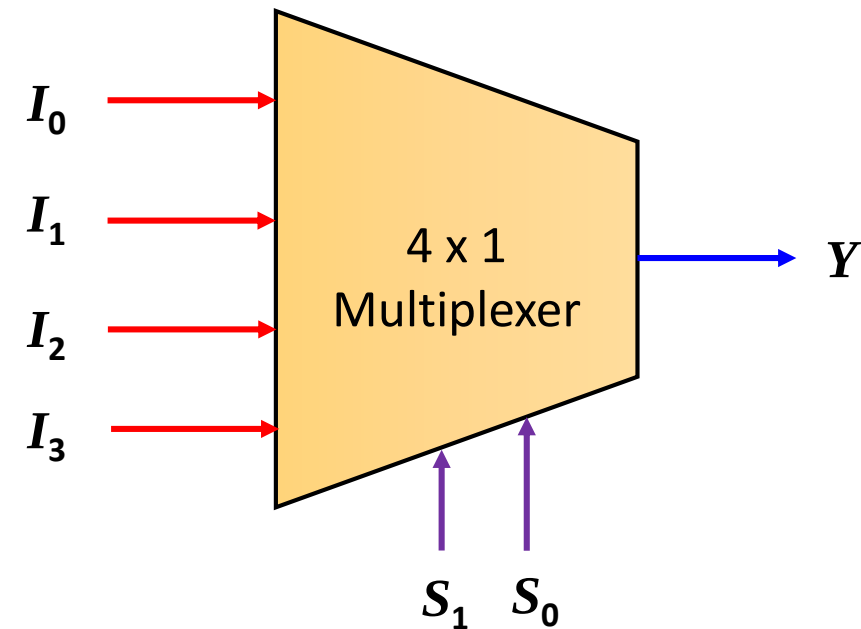
MULTIPLEXER

4x1 Multiplexer

- ✓ A **4x1 multiplexer** has **$4 = 2^2$ inputs and one output.**
- ✓ The input lines can be selected by **2 selection input S_1 and S_0 .**

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

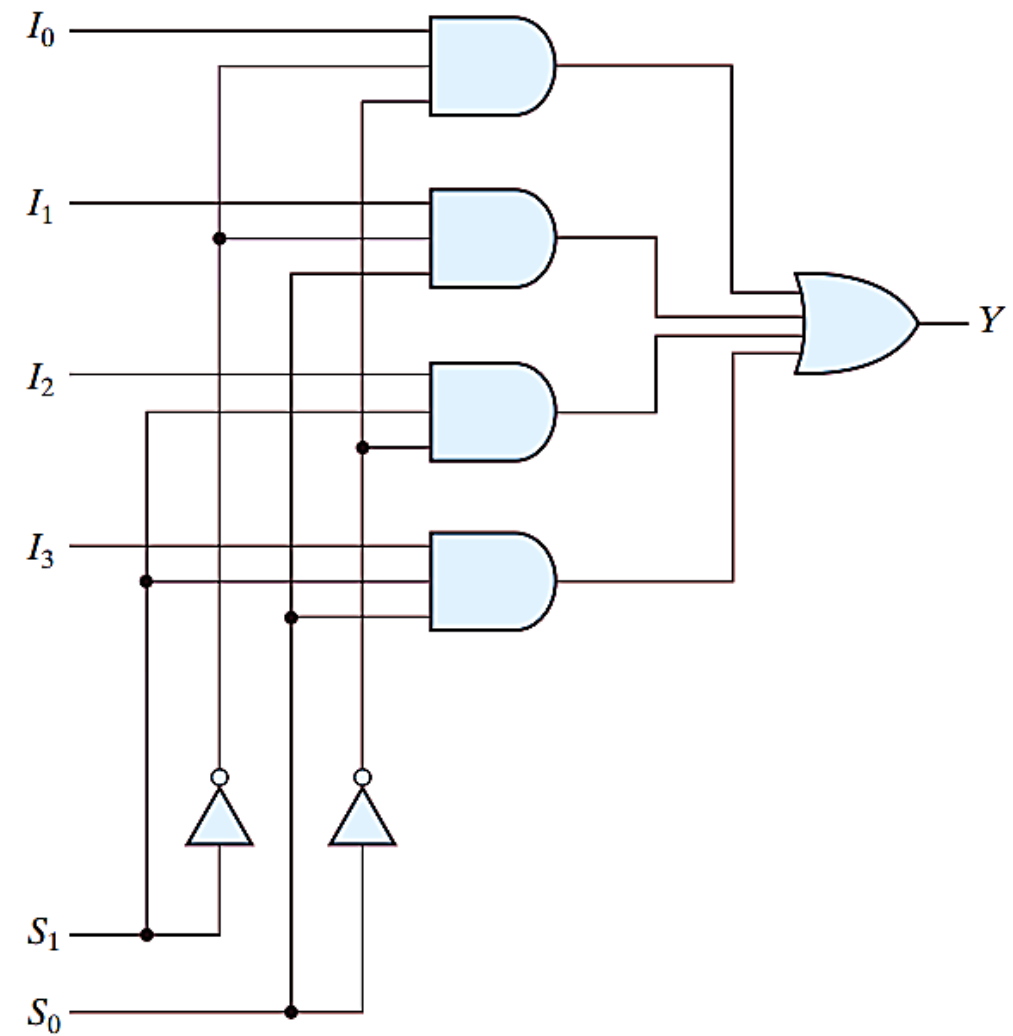
Function table



MULTIPLEXER

4x1 Multiplexer: Logic Diagram

$$Y = I_0 S_1' S_0' + I_1 S_1' S_0 + I_2 S_1 S_0' + I_3 S_1 S_0$$



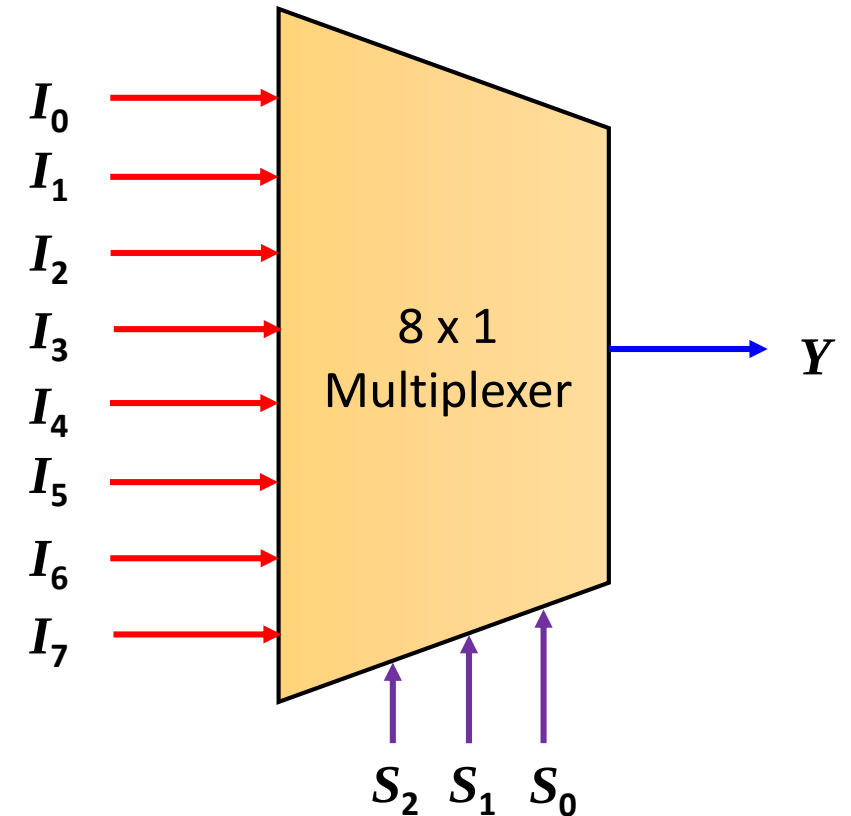
MULTIPLEXER

8x1 Multiplexer

- ✓ A **8x1 multiplexer** has **$8 = 2^3$** inputs and one output.
- ✓ The input lines can be selected by **3 selection input** **S_2, S_1 and S_0** .

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Function table

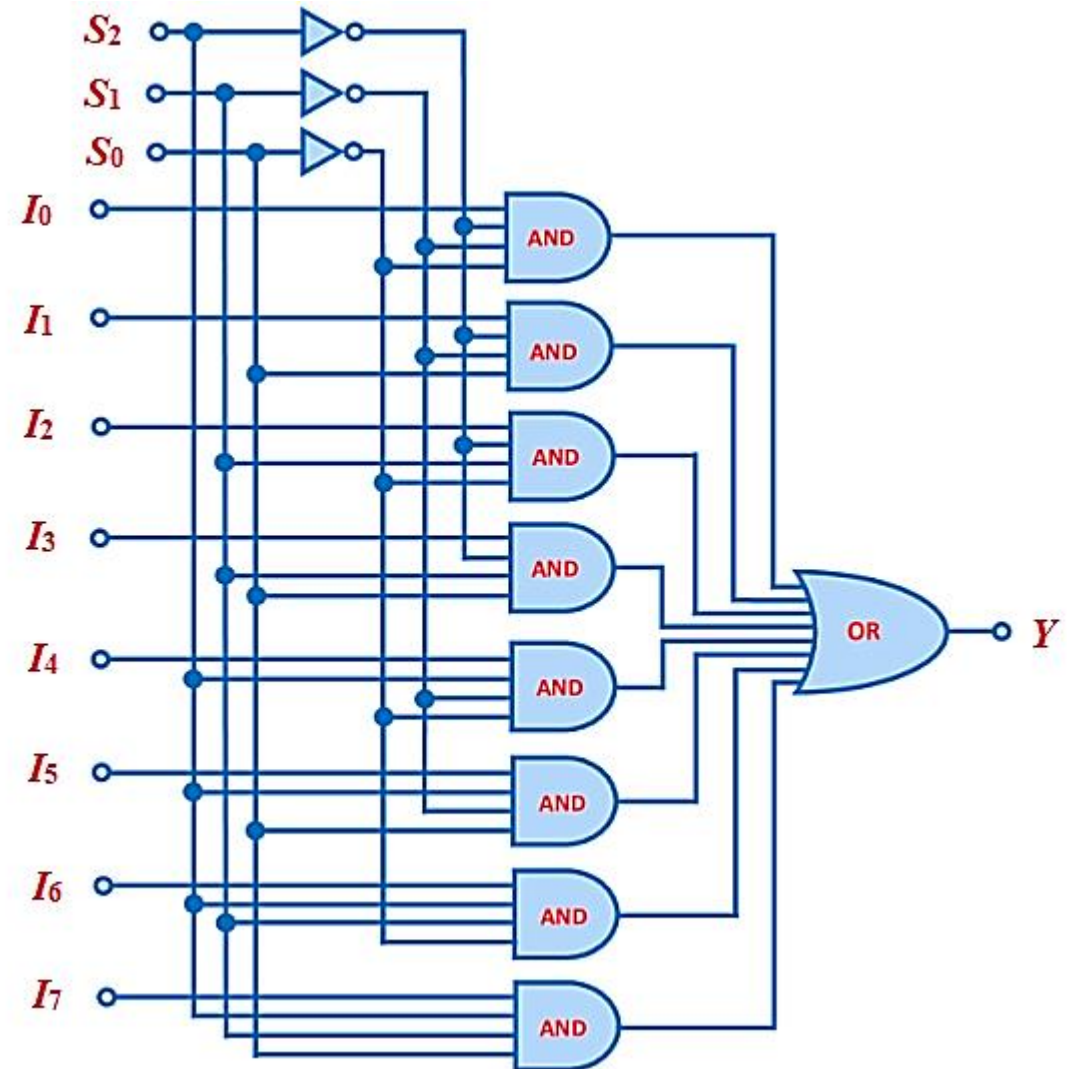


Block Diagram

MULTIPLEXER

8x1 Multiplexer: Logic Diagram

$$Y = I_0S_2'S_1'S_0' + I_1S_2'S_1'S_0 + I_2S_2'S_1S_0' + I_3S_2'S_1S_0 + I_4S_2S_1'S_0' + I_5S_2S_1'S_0 + I_6S_2S_1S_0' + I_7S_2S_1S_0$$



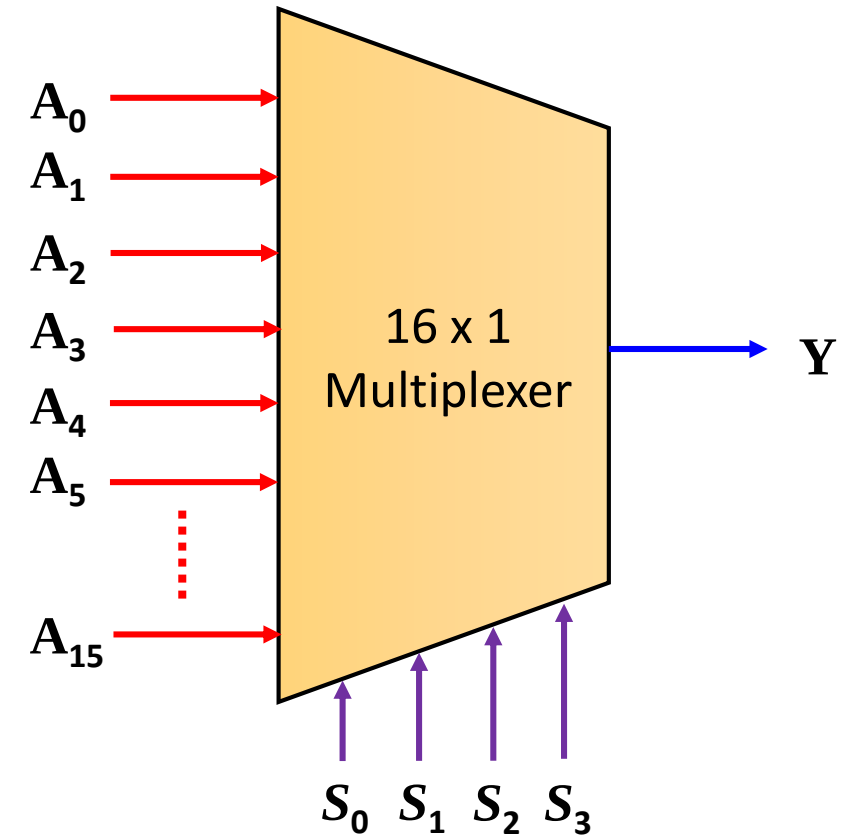
MULTIPLEXER

16x1 Multiplexer

- ✓ A **16x1 multiplexer** has **$16 = 2^4$** inputs and one output.
- ✓ The input lines can be selected by **4 selection input S_0, S_1, S_2 and S_3** .

INPUTS				Output
S_0	S_1	S_2	S_3	Y
0	0	0	0	A_0
0	0	0	1	A_1
0	0	1	0	A_2
0	0	1	1	A_3
0	1	0	0	A_4
0	1	0	1	A_5
0	1	1	0	A_6
0	1	1	1	A_7
1	0	0	0	A_8
1	0	0	1	A_9
1	0	1	0	A_{10}
1	0	1	1	A_{11}
1	1	0	0	A_{12}
1	1	0	1	A_{13}
1	1	1	0	A_{14}
1	1	1	1	A_{15}

Function table

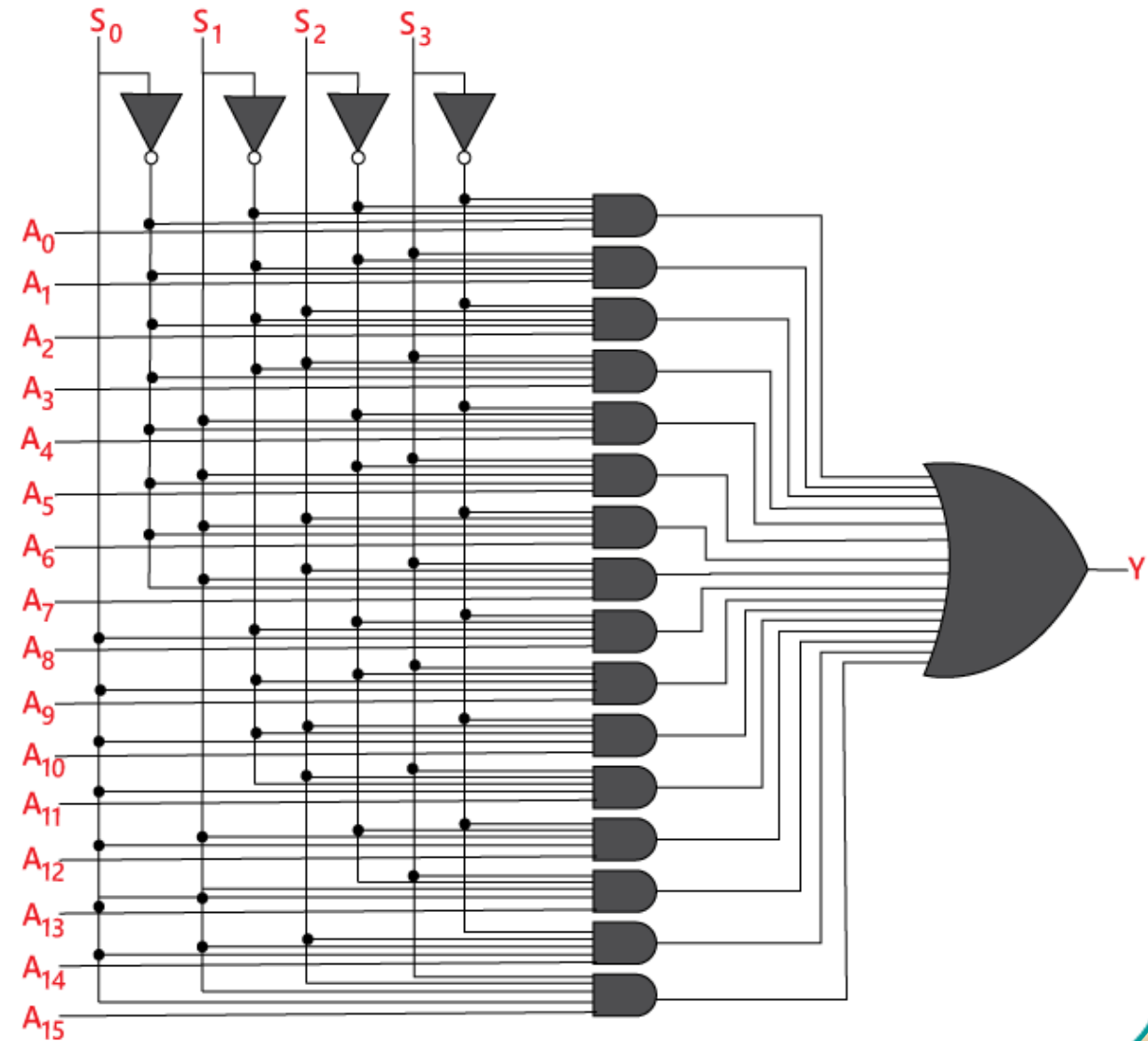


Block Diagram

MULTIPLEXER

16x1 Multiplexer: Logic Diagram

$$\begin{aligned} Y = & A_0 S'_0 S'_1 S'_2 S'_3 + A_1 S'_0 S'_1 S'_2 S_3 + A_2 S'_0 S'_1 S_2 S'_3 \\ & + A_3 S'_0 S'_1 S_2 S_3 + A_4 S'_0 S_1 S'_2 S'_3 + A_5 S'_0 S_1 S'_2 S_3 \\ & + A_6 S'_0 S_1 S_2 S'_3 + A_7 S'_0 S_1 S_2 S_3 + A_8 S_0 S'_1 S'_2 S'_3 \\ & + A_9 S_0 S'_1 S'_2 S_3 + A_{10} S_0 S'_1 S_2 S'_3 + A_{11} S_0 S'_1 S_2 S_3 \\ & + A_{12} S_0 S_1 S'_2 S'_3 + A_{13} S_0 S_1 S'_2 S_3 + A_{14} S_0 S_1 S_2 S'_3 \\ & + A_{15} S_0 S_1 S_2 S_3 \end{aligned}$$



Thank You